Causa ra?z y factores contribuyentes ? OrionOrders P1-8841

Fecha del incidente: 29 de septiembre de 2026 **Ventana de evidencia:** 09:30–10:41:30 **Estado de la causa raíz:** no confirmada formalmente **Alcance:** atribución únicamente con evidencia entregada; sin inferir hechos no observados.

1. Dictamen

La evidencia demuestra que el pool HTTP de `orders-api` hacia `customer-profile` se saturó en las muestras observadas y que la espera por adquirir conexión llegó a dominar el tiempo de varias solicitudes. La versión 4.18.0 documenta una reducción del máximo de conexiones de 200 a 20 antes del inicio del deterioro.

No hay evidencia suficiente para declarar que la reducción de 200 a 20 sea la causa raíz confirmada. Los artefactos no verifican qué valor fue cargado por cada pod, no cuantifican la demanda concurrente ni el presupuesto global de conexiones del servicio dependiente, y no incluyen una prueba controlada que aisle este cambio frente a la reestructuración del cliente HTTP u otros factores.

Por tanto, el dictamen preciso es:

> **Mecanismo técnico observado:** saturación del pool HTTP cliente de `orders-api` hacia `customer-profile`, con conexiones ocupadas al máximo, cola pendiente y espera de adquisición creciente.
>
> **Causa raíz candidata:** cambio introducido en 4.18.0, especialmente el máximo documentado de conexiones de 200 a 20. Es la hipótesis principal, temporal y funcionalmente consistente con las señales, pero no está probada como causa raíz.
>
> **Causa raíz confirmada:** no determinable con la evidencia actualmente disponible.

2. Hechos relevantes para la atribución

ID	Hecho observado	Fuente	Qué permite afirmar y qué no
F-01	Rollout 4.18.0 completado a las 09:30; primera alerta de latencia reportada a las 09:42.	`ORIONORDERS/05_DEVOPS_CICD_RELEASE/04_pipeline_4.18.0(3).log`; `ORIONORDERS/01_SOPORTE_INCIDENTE/02_P1-8841(2).txt`	Establece precedencia temporal; doce minutos de diferencia no prueban causalidad.
F-02	La nota de release indica reestructuración del cliente HTTP de customer-profile y cambio de `CUSTOMER_PROFILE_MAX_CONNECTIONS` de 200 a 20; connect/read timeouts quedan en 1000/2000 ms.	`ORIONORDERS/05_DEVOPS_CICD_RELEASE/03_RELEASE_4.18.0(2).md`	Prueba qué cambio documenta la release, no qué configuración efectiva tenía cada pod durante el incidente.
F-03	El archivo de configuración entregado declara `maxConnections: 20` y `pendingAcquireTimeoutMs: 5000`.	`ORIONORDERS/06_CONFIGURACION/12_http_client_config(2).yaml`	Evidencia de la configuración proporcionada; no contiene historial horario ni identificación de pods que la cargaron.

ID	Hecho observado	Fuente	Qué permite afirmar y qué no
F-04	El log registra aumento de `clientPoolWaitMs` desde las 09:42. A las 09:43:20 el pool informa `leased=20 max=20 pending=19`; a las 09:44:10, 20/20 y 25 pendientes.	`ORIONORDERS/02_APLICACION_DESARROLLO/05_orders-api(2).log`	Demuestra ocupación al máximo y cola en esas observaciones; no identifica por sí sola qué originó el aumento de demanda o retención.
F-05	A las 09:53:10 se registran 3549 ms de espera del pool y 118 ms de llamada customer-profile; a las 09:54:10, 4435 ms y 89 ms.	Log de orders-api citado arriba	En esas solicitudes, la espera local es mucho mayor que la duración downstream registrada; son muestras individuales.
F-06	APM muestra pool wait creciente desde las 09:42; los componentes de red y DB registrados son inferiores a la espera del pool en muestras citadas.	`ORIONORDERS/02_APLICACION_DESARROLLO/13_apm_trace_breakdown(2).csv`	Apoya que la espera local domina las trazas seleccionadas; no cubre todas las solicitudes.
F-07	Customer-profile presenta p95 agregado aproximado de 87–143 ms y errores inferiores a 0.4% en la ventana 09:30–10:15.	`ORIONORDERS/02_APLICACION_DESARROLLO/08_customer_profile_metrics(2).csv`	No respalda una degradación agregada sostenida en esa serie; no descarta latencias fuera de la ventana o variaciones por operación.
F-08	El balanceador sube de p95 0.7 s y 0.4% de 5xx a las 09:41 a p95 10.2 s y 38% de 5xx a las 10:15.	`ORIONORDERS/03_INFRAESTRUCTURA/10_load_balancer_metrics(2).csv`	Demuestra deterioro agregado hasta 10:15; no identifica una causa por sí solo y no hay serie posterior.
F-09	Los primeros fallos de liveness/terminaciones aparecen a las 09:45:12–09:45:14, después del inicio de síntomas; hay más eventos posteriores.	`ORIONORDERS/03_INFRAESTRUCTURA/07_k8s_events(2).log`	Hace improbable que esos eventos registrados iniciaran la espera a las 09:42; no demuestra el mecanismo del primer deterioro.
F-10	La arquitectura indica que liveness comparte servidor HTTP/grupo de hilos con tráfico de negocio; la sonda tiene timeout de 1 s y umbral de 3 fallos.	`ORIONORDERS/00_DOCUMENTACION_GENERAL/01_ARQUITECTURA(3).md`; `ORIONORDERS/03_INFRAESTRUCTURA/11_health_probe_config(2).yaml`	Hace plausible una interacción entre contención y fallos de sonda; falta telemetría de hilos que pruebe esa interacción en este incidente.
F-11	El HPA reporta tamaño deseado 18 a las 09:52; pods listos disminuyen en el CSV hasta 4 en la muestra de 10:08.	Eventos Kubernetes y `ORIONORDERS/03_INFRAESTRUCTURA/06_k8s_pod_metrics(2).csv`	Indica objetivo de escalado y capacidad lista observada; no demuestra que el escalado causara o resolviera la saturación.
F-12	Alertas DB de 92–96% conexiones y CPU 81% son de `db-reporting-01`. El datasource de orders-api apunta a `orders-primary-01`, que a las 10:02 tenía 118/300 conexiones, 44% CPU y cero lock waiters.	`ORIONORDERS/04_BASE_DE_DATOS/09_db_alerts(2).txt`	No sustenta saturación de la DB primaria en esa instantánea; la lectura no excluye problemas en otros momentos.
F-13	El pipeline pasa sus etapas registradas, pero no valida rango min/max de `CUSTOMER_PROFILE_MAX_CONNECTIONS` ni ejecuta carga similar a producción.	`ORIONORDERS/05_DEVOPS_CICD_RELEASE/04_pipeline_4.18.0(3).log`	Demuestra brechas de prevención/detección del cambio; no demuestra que esas brechas generaran directamente la saturación.
F-14	El log de aplicación termina a las 10:41:30 con espera de 9000 ms, 128 pendientes y status 200 en la solicitud final.	Log de orders-api	La degradación seguía presente en esa muestra; el status 200 no prueba recuperación. No se conoce el final del incidente.

ID	Hecho observado	Fuente	Qué permite afirmar y qué no
F-15	No hay conciliación de órdenes ni evidencia posterior suficiente para verificar pérdida/cero pérdida.	Ficha P1 y conjunto de artefactos suministrado	Integridad completa de pedidos: no verificable. La ausencia de evidencia inicial de pérdida no es prueba de ausencia de pérdida.

3. Causa raíz candidata: evaluación basada en evidencia

Hipótesis CR-01: reducción del límite del pool en 4.18.0

Mecanismo propuesto: con un máximo menor de conexiones HTTP por instancia, la concurrencia de solicitudes síncronas podría superar la capacidad disponible. Solicitudes que no obtienen conexión esperarían en la cola de adquisición, elevando la latencia de `orders-api` y eventualmente los errores.

Elemento	Evaluación
Secuencia temporal	El rollout termina a las 09:30; la primera alerta aparece a las 09:42 y las primeras mediciones de espera crecen a las 09:42. Compatible, no concluyente.
Cambio documentado	Máximo 200→20 y cliente HTTP reestructurado. Hecho documentado en nota de release.
Mecanismo observable	Sí: 20 conexiones ocupadas de 20, pendientes y espera de pool de cientos a miles de milisegundos.
Relación con la experiencia	Sí: el p95 del balanceador sube, y solicitudes muestreadas muestran espera local mucho mayor que llamada downstream.
Evidencia alternativa	La telemetría disponible no respalda que la DB primaria estuviera saturada a las 10:02. Customer-profile agregado no muestra degradación sostenida importante en el periodo que cubre, aunque no es una exclusión total.
Prueba de causalidad	No disponible: no hay configuración efectiva por pod, demanda concurrente completa, capacidad global downstream ni comparación controlada.
Clasificación	**Hipótesis principal; causa raíz no confirmada.**
Confianza	**Alta** en la saturación observada; **moderada-alta** en el cambio como causa probable, condicionada a verificar configuración y capacidad.

No se concluye que «20 sea incorrecto» en abstracto: el límite seguro depende del patrón de concurrencia, número de pods, comportamiento real del cliente y presupuesto global de conexiones aceptado por `customer-profile`. Tampoco se concluye que restaurar 200 por pod sea seguro.

4. Factores contribuyentes sustentados o plausibles

FC-01 — Reestructuración del cliente HTTP y límite documentado

- **Evidencia:** la release de 4.18.0 documenta ambas modificaciones; el archivo

de configuración entregado declara máximo 20; el pool se observa al máximo y con cola.

- **Lectura:** el cambio es parte de la hipótesis causal principal, no un factor

independiente confirmado. La reestructuración podría afectar el ciclo de adquisición/liberación, pero no se aportó diff de código ni traza que demuestre un defecto en ese ciclo.

- **Estado:** límite y reestructuración documentados; mecanismo exacto por

verificar.

FC-02 — Liveness acoplado al servidor de solicitudes

- **Evidencia:** arquitectura comparte servidor HTTP/grupo de hilos; hay

timeouts de liveness y terminaciones desde 09:45.

- **Lectura:** el acoplamiento hace plausible que contención de recursos afecte

sondas y que las terminaciones reduzcan capacidad. Como los eventos registrados empiezan después de la espera inicial, son más compatibles con amplificación que con iniciación.

- **Estado:** componente arquitectónico y eventos comprobados; contribución

causal cuantitativa no probada.

FC-03 — Disminución de pods listos durante la degradación

- **Evidencia:** CSV pasa de 12 listos al comienzo a 4 en muestras tardías;

eventos registran terminaciones por liveness.

- **Lectura:** menor capacidad lista coincide temporalmente con empeoramiento.

No permite establecer cuánto de la reducción se debe a liveness, ni cuánto empeoró por ella la cola del pool.

- **Estado:** descenso observado; amplificación plausible. El contador

`cumulative_restarts` no es monotónico y no debe usarse como total fiable.

FC-04 — Validaciones insuficientes en el pipeline

- **Evidencia:** pipeline sin rango aprobado para el máximo y sin carga

representativa; schema valida solo tipo y presencia.

- **Lectura:** brecha de prevención/detección que pudo permitir que un cambio de

configuración llegara a producción sin descubrir su efecto bajo concurrencia. No es por sí misma una causa técnica que explique las esperas.

- **Estado:** brecha confirmada; relación de contribución a la ocurrencia

indirecta y no demostrada.

FC-05 — Ausencia de observabilidad/alertas específicas del pool

- **Evidencia:** SLO vigente no incluye alertas específicas para espera de

adquisición, saturación de conexiones HTTP o espera interna del cliente.

- **Lectura:** puede retrasar la detección y dificultar el diagnóstico; no

provoca por sí sola la ocupación de 20/20.

- **Estado:** brecha de detección documentada; duración real atribuible no

disponible.

FC-06 — Confusión inicial por alarma de otra base de datos

- **Evidencia:** la alerta nombra `db-reporting-01`; el datasource del servicio

usa `orders-primary-01`; chat corrige la diferencia a las 09:54.

- **Lectura:** posible desvío de atención durante la investigación, no causa

técnica de latencia ni saturación del pool.

- **Estado:** discrepancia de host confirmada; impacto en tiempo de respuesta no

cuantificado.

5. Factores no sustentados como causa raíz

Propuesta de causa	Evidencia disponible	Determinación
Saturación de `orders-primary-01`	La alarma DB pertenece a reporting. La primaria informa 118/300 conexiones, CPU 44% y cero lock waiters a las 10:02; APM muestra DB menor que pool wait en trazas citadas.	**No sustentada** para la primaria con la evidencia actual. No excluye incidentes en otros instantes.
Liveness inició el incidente	La primera latencia/espera aparece a las 09:42; primer evento liveness a las 09:45:12.	**Contradice la secuencia temporal** para los eventos observados; posible amplificación.
Customer-profile estaba globalmente caído	Métricas agregadas disponibles muestran p95 ~87–143 ms y errores <0.4% hasta 10:15; las solicitudes citadas suelen medir downstream decenas/pocos cientos de ms.	**No demostrado.** La cobertura no permite descartar episodios por operación o después de 10:15.
El HPA corrigió o provocó la saturación	Evento registra tamaño deseado 18, pero los pods listos no se recuperan de inmediato y siguen bajando en muestras.	**No demostrado** para resolución ni para causalidad inicial.
Se perdieron o no se perdieron pedidos	No existe conciliación completa.	**No verificable.** No afirmar pérdida confirmada ni integridad total.

6. Criterio para confirmar la causa raíz

La hipótesis CR-01 solo debería elevarse a causa confirmada si se obtiene, como mínimo:

1. Historial temporal de Helm/ConfigMap y valor realmente cargado por pod.
2. Métrica de concurrencia por pod y del pool (ocupadas, libres, pendientes,

espera, timeouts y duración de conexiones arrendadas).

1. Presupuesto total y capacidad acordada de customer-profile, incluidos otros

consumidores y el máximo global posible entre réplicas.

1. Correlación de trazas por solicitud/pod que descarte otras esperas o

retenciones no visibles en las muestras.

1. Prueba controlada en ambiente seguro, con carga comparable y cambio de una

sola variable, o evidencia operacional equivalente con antes/después verificable.

1. Telemetría posterior que compruebe recuperación sostenida; conciliación de

órdenes separada para determinar integridad.

La prueba debe respetar las restricciones del caso: no desactivar autenticación, no reiniciar la base de datos, no cambiar datos directamente y preservar las órdenes ya confirmadas. Cualquier intervención productiva requiere aprobación operativa expresa y evaluación de capacidad downstream.

7. Conclusión final

Causa raíz formal: no confirmada con los artefactos disponibles. **Mecanismo inmediato demostrado:** pool HTTP cliente observado en su máximo, con cola y espera de adquisición creciente. **Causa candidata principal:** cambio de 4.18.0 (máximo documentado 200→20 y reestructuración del cliente HTTP), con confianza moderada-alta, pendiente de configuración efectiva y prueba causal. **Factores contribuyentes posibles:** liveness compartido, disminución de pods listos, gates de CI/CD sin rango/carga, falta de alertas del pool y desvío inicial por la alarma de otro host. **Recuperación e integridad de pedidos:** no verificables.

8. Fuentes

- `ORIONORDERS/00_DOCUMENTACION_GENERAL/01_ARQUITECTURA(3).md`
- `ORIONORDERS/01_SOPORTE_INCIDENTE/02_P1-8841(2).txt`
- `ORIONORDERS/01_SOPORTE_INCIDENTE/15_current_slo(2).md`
- `ORIONORDERS/01_SOPORTE_INCIDENTE/16_chat_crisis(2).txt`
- `ORIONORDERS/02_APLICACION_DESARROLLO/05_orders-api(2).log`
- `ORIONORDERS/02_APLICACION_DESARROLLO/08_customer_profile_metrics(2).csv`
- `ORIONORDERS/02_APLICACION_DESARROLLO/13_apm_trace_breakdown(2).csv`
- `ORIONORDERS/03_INFRAESTRUCTURA/06_k8s_pod_metrics(2).csv`
- `ORIONORDERS/03_INFRAESTRUCTURA/07_k8s_events(2).log`
- `ORIONORDERS/03_INFRAESTRUCTURA/10_load_balancer_metrics(2).csv`
- `ORIONORDERS/03_INFRAESTRUCTURA/11_health_probe_config(2).yaml`
- `ORIONORDERS/04_BASE_DE_DATOS/09_db_alerts(2).txt`
- `ORIONORDERS/05_DEVOPS_CICD_RELEASE/03_RELEASE_4.18.0(2).md`
- `ORIONORDERS/05_DEVOPS_CICD_RELEASE/04_pipeline_4.18.0(3).log`
- `ORIONORDERS/06_CONFIGURACION/12_http_client_config(2).yaml`
- `ORIONORDERS/07_SEGURIDAD/17_security_notes(2).md`
